Probabilistic Robotics

Mihir Rao

Based on Probabilistic Robotics by Thrun, Burgard & Fox

Contents

1 Introduction **2**
- 1.1 Why uncertainty is unavoidable . . . 2
- 1.2 Represent a belief, not just a best guess . . . 2
- 1.3 Global localization: the hallway example . . . 3
- 1.4 Uncertainty should influence action . . . 4
- 1.5 Benefits and costs of the probabilistic approach . . . 4
- 1.6 How the ideas fit together . . . 5

2 Recursive State Estimation **6**
- 2.1 Probability as a language for uncertainty . . . 6
- 2.2 Moments, Gaussians, and entropy . . . 7
- 2.3 State, controls, and observations . . . 8
- 2.4 The recursive Bayes filter . . . 9
- 2.5 Worked example: estimating a door's state . . . 10
- 2.6 Modeling and computational limits . . . 11

3 Gaussian Filters **12**
- 3.1 What a Gaussian belief represents . . . 12
- 3.2 The linear Gaussian model . . . 12
- 3.3 The Kalman filter update . . . 13
- 3.4 Worked example: one-dimensional tracking . . . 15
- 3.5 The extended Kalman filter . . . 15
- 3.6 The information filter . . . 17
- 3.7 The extended information filter . . . 18
- 3.8 Computational and modeling tradeoffs . . . 19

4 Nonparametric Filters **21**
- 4.1 Discrete Bayes filters . . . 21
- 4.2 Histograms for continuous states . . . 22
- 4.3 Static binary state and log odds . . . 23
- 4.4 Particle beliefs and the bootstrap filter . . . 24
- 4.5 Why importance weights work . . . 25
- 4.6 Systematic resampling . . . 26
- 4.7 Failure modes and resource choices . . . 28

Chapter 1: Introduction

A robot must perceive and act in a physical world it does not fully know. The central idea of PROBABILISTIC ROBOTICS is to represent that uncertainty explicitly and use it when interpreting observations and choosing actions. The aim is reliable behavior despite incomplete information, rather than pretending that a single estimate is certainly correct.

1.1 Why uncertainty is unavoidable

A factory can constrain where objects are placed and how a task is performed. A home or road is less controlled: people move, objects change, and unexpected situations arise. Five sources of uncertainty matter even when the software executes exactly as intended.

1. **The environment.** The world changes in ways the robot cannot fully predict, such as a person stepping into its path.
2. **Sensors.** Limited range and resolution hide information; measurement noise perturbs what is visible. A camera cannot see through a wall, and seeing a door does not identify which of several similar doors it is.
3. **Actuation.** A commanded movement is not an exact displacement. Wheel slip, control noise, and mechanical wear change the outcome.
4. **Models.** A model simplifies the underlying physics. Even a perfect calculation with an imperfect model can give an inaccurate prediction.
5. **Computation.** A robot must respond on time. An approximation that finishes before the next decision may be more useful than an exact calculation that arrives too late.

Better hardware can reduce some errors, but it cannot eliminate missing information, model simplifications, or ambiguity. Robust software must account for these limitations.

1.2 Represent a belief, not just a best guess

Definition 1.1 [BELIEF]. A BELIEF is a probability distribution over possible states of the world, given the robot's information. For localization, the hypotheses are possible robot locations in a known map. Write this distribution as $\mathrm{bel}(x)$.

In a discrete model, probabilities sum to one. For a continuous position, bel(x) is a density: probability is area under its curve, not the curve's height at one point. A belief can have several separated peaks, preserving different explanations of the same sensor reading.

For example, a robot may be either near the kitchen or near the bedroom because the two doorways look alike. Choosing one location immediately loses the alternative. Keeping both hypotheses lets later observations resolve the ambiguity. This uncertainty is different from a small error around one known doorway; one narrow peak cannot describe both situations.

1.3 Global localization: the hallway example

Localization estimates a robot's position in an external reference frame using sensor data and a map. In global localization, its starting location is unknown. Consider a hallway with three indistinguishable doors.

1. **Start with a prior.** With no reason to favor one position over another, use a uniform belief over the allowed hallway positions.
2. **Observe a door.** Locations near any of the three doors become more plausible. Other positions retain some probability if the detector can make a false report. One observation does not select a unique door.
3. **Move.** Shift the hypotheses by the estimated displacement and spread them to account for motion error. Uncertain motion generally broadens the possible current locations.
4. **Observe another door.** Favor hypotheses consistent with both the movement and the new reading. If only one door-to-door spacing fits well, one peak becomes dominant. In a repetitive hallway, ambiguity may remain.

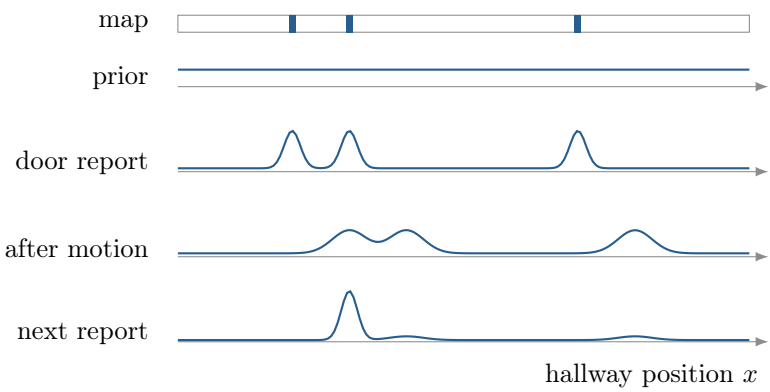


Figure 1: A schematic belief sequence. Motion shifts the three hypotheses to the right and broadens them; the next door report favors one. Heights have no numerical scale, and the rows are not a computed filtering run.

1.3.1 A small numerical illustration

Take four possible positions: three door locations A, B, C and a non-door location N. Give each prior probability 1/4. Suppose a detector reports "door" with probability 9/10 at a door and 1/10 at N. Reweight each prior by the likelihood of the report:

position	A	B	C	N
prior	1/4	1/4	1/4	1/4
likelihood	9/10	9/10	9/10	1/10
product	9/40	9/40	9/40	1/40
posterior	9/28	9/28	9/28	1/28

The products sum to 28/40, so dividing by that sum produces a distribution. The report strongly favors being at *some* door, but each individual door still has probability only 9/28. It also leaves a small chance that the report was wrong. These invented numbers illustrate the reweighting; the general recursive Bayes filter is developed in Chapter 2.

1.4 Uncertainty should influence action

Perception estimates what is happening; action changes the world and what can be learned about it. A robot uncertain between two locations may move toward a distinctive landmark before committing to a route. That action has an information benefit even if it does not immediately shorten the journey. This creates a tradeoff between EXPLORATION, gathering information, and EXPLOITATION, using current knowledge to accomplish the task.

A decision should therefore use the relevant belief, not just its most likely state. Probabilistic representation alone does not select an action: a task objective and a model of possible consequences are also needed.

1.5 Benefits and costs of the probabilistic approach

Explicit uncertainty gives a common language for combining imperfect models with noisy measurements. It supports robust localization, recovery after a robot is moved unexpectedly, and map building without a reliable global position reference. Learning an unknown model can likewise be viewed as estimating persistent unknown quantities from accumulated data.

These are capabilities of suitable algorithms, not guarantees from the word "probabilistic." A model that rules out the true location cannot recover it merely by normalizing its remaining hypotheses. Incorrect assumptions or unjustifiably small uncertainty can still produce confident mistakes.

The main cost is representing and updating a distribution. A general density over a continuous state space cannot be stored exactly in a finite table. Compact representations such as Gaussians,

discretizations, and sample-based approximations trade expressive power against memory and computation. A single Gaussian is compact, for example, but cannot preserve three distinct door hypotheses. The representation must fit both the uncertainty and the available computation.

1.6 How the ideas fit together

Classical approaches emphasize planning with models; reactive approaches respond directly to sensing; hybrid systems combine fast reactions with higher-level planning. The probabilistic approach combines models and perception through explicit uncertainty at either level. Randomized search is a separate idea: an algorithm can use random numbers to search a known world without representing uncertainty about that world. Probabilistic estimation also has roots in Kalman filtering and early joint localization and mapping methods.

The progression is from state-estimation filters to motion and sensor models, then localization and mapping, and finally action selection under uncertainty. Understanding an algorithm requires its derivation, executable procedure, implementation limitations, and experimental behavior. A correct equation alone does not establish that a robot implementation behaves reliably.

Source. Sebastian Thrun, Wolfram Burgard, and Dieter Fox, Probabilistic Robotics, early draft, Chapter 1, pp. 1–8 (PDF pp. 13–20).

Chapter 2: Recursive State Estimation

State estimation reconstructs quantities that sensors do not reveal directly. A camera records an image, for example, rather than the robot's coordinates. The aim is to maintain a distribution over the possible state as controls and measurements arrive. A Bayes filter does this recursively: yesterday's belief becomes the starting point for today's calculation.

2.1 Probability as a language for uncertainty

Write X for a random variable and x for one of its possible values. The abbreviation $p(x)$ means $P(X = x)$ for a discrete variable, or a density at x for a continuous variable. These are different objects:

$$\sum_x p(x) = 1, \qquad \int p(x)\,dx = 1, \qquad P(a \le X \le b) = \int_a^b p(x)\,dx.$$

Both are nonnegative, but a density can exceed one. A uniform position on a 0.1-meter interval has density 10 per meter; the area, $10(0.1) = 1$, is the probability. A single point has zero probability under a continuous density.

A joint distribution $p(x, y)$ describes two quantities together. Conditioning fixes information that is already known, and marginalization removes a quantity by considering all of its possible values:

$$p(x, y) = p(x \mid y)p(y), \qquad p(x \mid y) = \frac{p(x, y)}{p(y)},$$
$$p(x) = \sum_y p(x, y) = \sum_y p(x \mid y)p(y).$$

Use integrals instead of sums for continuous variables. The last equality is the law of total probability: average the conditional answers using the probabilities of the conditions. Conditional densities are understood on their support; the displayed discrete ratio requires $p(y) > 0$.

Variables are independent when $p(x, y) = p(x)p(y)$, equivalently $p(x \mid y) = p(x)$ wherever the conditional is defined: learning Y then does not change the distribution of X.

Theorem 2.1 [Bayes' rule]. *For evidence z with positive probability or density,*

$$p(x \mid z) = \frac{p(z \mid x)p(x)}{p(z)} = \eta\, p(z \mid x)p(x).$$

Here $p(x)$ is the prior, *$p(x \mid z)$ the* posterior, *and $p(z \mid x)$ the* likelihood *of the observed data under hypothesis x.*

Proof. The product rule gives both $p(x, z) = p(x \mid z)p(z)$ and $p(x, z) = p(z \mid x)p(x)$. Equate them and divide by $p(z)$. □

The likelihood answers "Would this state produce the reading?" It is not the probability of the state given that reading. To compute the posterior, multiply prior and likelihood at each state, then divide by the sum or integral of those products. The normalizer η is constant with respect to x, although it changes when the data change.

2.2 Moments, Gaussians, and entropy

The EXPECTATION is a probability-weighted average:

$$\mu = \mathbb{E}[X] = \sum_x xp(x) \quad \text{or} \quad \mu = \int xp(x)\,dx.$$

When the moments exist, $\mathbb{E}[aX + b] = a\mathbb{E}[X] + b$. For a vector X, its COVARIANCE is

$$\Sigma = \mathbb{E}[(X - \mu)(X - \mu)^T].$$

Diagonal entries are variances; off-diagonal entries describe how deviations in two coordinates vary together. In one dimension, $\sigma^2 = \mathbb{E}[(X - \mu)^2] = \mathbb{E}[X^2] - \mu^2$. Variance uses squared units: a position standard deviation of 2 meters means variance 4 square meters.

A one-dimensional GAUSSIAN has density

$$\mathcal{N}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(x - \mu)^2}{2\sigma^2}\right].$$

For $x \in \mathbb{R}^d$, the corresponding multivariate density is

$$\mathcal{N}(x; \mu, \Sigma) = \frac{\exp[-\frac{1}{2}(x - \mu)^T\Sigma^{-1}(x - \mu)]}{(2\pi)^{d/2}\det(\Sigma)^{1/2}}.$$

This density formula assumes Σ is positive definite. A singular covariance describes a degenerate Gaussian supported on a lower-dimensional set, for which the ordinary full-dimensional density formula does not apply.

For a discrete distribution, ENTROPY measures average surprise:

$$H(X) = -\sum_x p(x)\log_2 p(x), \qquad 0\log_2 0 := 0.$$

A fair binary variable has entropy 1 bit; probabilities $(0.9, 0.1)$ give about 0.469 bits. A certain outcome has entropy zero. Entropy concerns the whole distribution, so it can describe uncertainty that a single variance or best guess misses. It will later help quantify information-gathering actions.

2.3 State, controls, and observations

The STATE collects the quantities needed to predict the future. It can include robot pose, joint angles, velocity, landmarks, moving people, sensor health, and battery charge. A planar pose is (x, y, θ); position alone is insufficient to predict motion when heading or velocity is unknown. Walls may be static while the robot moves. A state containing continuous pose and a discrete sensor-failure flag is a HYBRID STATE.

The basic notation is:

Symbol	Meaning
x_t	State at time t
u_t	Control information for the interval $(t-1, t]$
z_t	Measurement associated with state x_t
$z_{1:t}, u_{1:t}$	Measurement and control histories through t

Odometry is physically sensed, but it is often treated as control information because it describes a change in pose. "Do nothing" is also a control: time passes and the environment can still change. The discrete ordering control-then-measurement is a model of the data, rather than a claim that real sensing and motion occur in separate phases. Delayed observations must be associated with the time they describe.

2.3.1 The Markov model

A state is COMPLETE if knowing it makes earlier history unnecessary for predicting subsequent events. The filtering model assumes

$$p(x_t \mid x_{0:t-1}, z_{1:t-1}, u_{1:t}) = p(x_t \mid x_{t-1}, u_t),$$
$$p(z_t \mid x_{0:t}, z_{1:t-1}, u_{1:t}) = p(z_t \mid x_t).$$

The first law is the TRANSITION MODEL; the second is the MEASUREMENT MODEL. With an initial law $p(x_0)$, they specify a controlled hidden Markov model. They say what a state would generate, which is why they are called generative models.

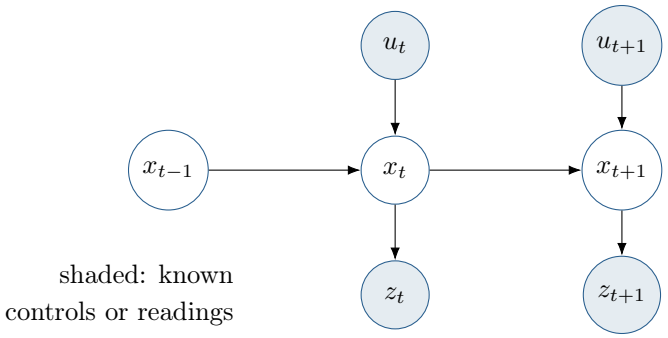


Figure 2: A state mediates the effect of earlier history. Controls influence transitions; readings provide evidence about the state that produced them.

These are CONDITIONAL INDEPENDENCE assumptions. In general, $X \perp Y \mid S$ means $p(x, y \mid s) = p(x \mid s)p(y \mid s)$, not that X and Y are independent without S. Two sensors may agree because they observe the same unknown state, yet have independent errors given that state. The converse implication also fails: independent fair bits become dependent if their parity is revealed, since knowing one then determines the other.

2.4 The recursive Bayes filter

Use $b_t(x_t) = \text{bel}(x_t)$ as shorthand for the filtered belief, and a bar for the prediction before the newest reading:

$$b_t(x_t) = p(x_t \mid z_{1:t}, u_{1:t}),$$
$$\bar{b}_t(x_t) = p(x_t \mid z_{1:t-1}, u_{1:t}).$$

The update has two operations:

$$\bar{b}_t(x_t) = \int p(x_t \mid x_{t-1}, u_t) b_{t-1}(x_{t-1})\, dx_{t-1}, \tag{2.1}$$
$$b_t(x_t) = \eta\, p(z_t \mid x_t) \bar{b}_t(x_t). \tag{2.2}$$

Prediction averages possible transitions over every plausible previous state. Correction reweights each current hypothesis by how well it explains the new reading. Think of prediction as spreading yesterday's possibilities through the motion model, and correction as testing the resulting possibilities against the sensor.

2.4.1 Why the recursion is correct

Let $\mathcal{H}_{t-1} = (z_{1:t-1}, u_{1:t-1})$. Assume also that the selected control conveys no additional hidden-state information once this history is known. This holds, for example, when the controller chooses its action using only the recorded history and independent randomization.

1. **Apply Bayes' rule to the latest reading.**

$$p(x_t \mid \mathcal{H}_{t-1}, u_t, z_t) = \eta\, p(z_t \mid x_t, \mathcal{H}_{t-1}, u_t) p(x_t \mid \mathcal{H}_{t-1}, u_t).$$

   The measurement assumption reduces the first factor to $p(z_t \mid x_t)$. This proves the correction equation.

2. **Marginalize the previous state.**

$$\bar{b}_t(x_t) = \int p(x_t \mid x_{t-1}, \mathcal{H}_{t-1}, u_t) p(x_{t-1} \mid \mathcal{H}_{t-1}, u_t)\, dx_{t-1}.$$

   The transition assumption reduces the first factor to $p(x_t \mid x_{t-1}, u_t)$. The control assumption reduces the second to $b_{t-1}(x_{t-1})$. This proves prediction.

3. **Initialize and repeat.** Starting with the correct prior b_0, these two identities give the correct posterior at every step by induction. The stored belief is therefore a sufficient summary of the history for this filtering calculation.

Algorithm 2.1: One Bayes-filter update

Input: Previous belief b_{t-1}, control u_t, reading z_t
for each candidate state x_t **do**
  $\bar{b}_t(x_t) \leftarrow \int p(x_t \mid x_{t-1}, u_t) b_{t-1}(x_{t-1})\, dx_{t-1}$
  $q_t(x_t) \leftarrow p(z_t \mid x_t) \bar{b}_t(x_t)$
$s \leftarrow \int q_t(x_t)\, dx_t$
return $b_t = q_t / s$

Replace integrals by sums for discrete states. The procedure requires $s > 0$; if all hypotheses give zero likelihood, normalization cannot repair the inconsistent model or data. For a continuous space, "each candidate state" is a mathematical specification, not a literal finite loop.

Choose b_0 to express the initial knowledge: a point mass for certainty, a localized distribution for approximate knowledge, or a uniform distribution on a bounded feasible region for global uncertainty. There is no proper uniform probability distribution over the entire real line.

2.5 Worked example: estimating a door's state

The door is either open (O) or closed (C). Initially, $b_0(O) = b_0(C) = 1/2$. For a report $z =$ "open", the sensor model is

$$p(z \mid O) = 0.6, \qquad p(z \mid C) = 0.2.$$

The complementary "closed" probabilities are 0.4 and 0.8 respectively. Waiting leaves the door unchanged. Pushing leaves an open door open, and opens a closed door with probability 0.8.

1. **Wait, then receive an "open" report.** Prediction leaves $(1/2, 1/2)$ unchanged. The unnormalized weights are $(0.6/2, 0.2/2) = (0.3, 0.1)$, which sum to 0.4. Thus

$$b_1(O) = 0.3/0.4 = 3/4, \qquad b_1(C) = 1/4.$$

2. **Push the door.** The next open-state prediction includes both an already-open door and a successfully opened closed door:

$$\bar{b}_2(O) = 1(3/4) + 0.8(1/4) = 0.95, \qquad \bar{b}_2(C) = 0.2(1/4) = 0.05.$$

3. **Receive another "open" report.** The new weights are $(0.6 \cdot 0.95, 0.2 \cdot 0.05) = (0.57, 0.01)$, giving

$$b_2(O) = \frac{57}{58} \approx 0.9828, \qquad b_2(C) = \frac{1}{58}.$$

The remaining closed-door probability is small, but it can matter if moving through a closed door is costly. This posterior describes the *current* door state; it does not assert that both earlier sensor reports were correct. The push could have changed the state between them.

2.6 Modeling and computational limits

Practical state is almost always incomplete. A constant sensor bias, an unmodeled moving person, or a persistent controller variable can correlate events that the model treats as independent. Repeatedly counting the same biased reading as fresh evidence can then produce unjustified confidence. Adding a bias or velocity variable to the state can make the model more appropriate, at the cost of a larger inference problem. Approximate belief representations and inaccurate transition or sensor models introduce further errors; they do not change the underlying Bayes identities.

Choosing an implementation trades off three things:

- **Accuracy.** A single Gaussian cannot preserve separated door hypotheses; a coarse grid cannot resolve positions within a cell.
- **Computation.** Closed-form Gaussian updates can be compact. Fine grids grow rapidly with dimension; particle counts trade runtime for sampling accuracy.
- **Implementation.** Some models allow easy simulation but difficult integration, favoring particles. Other models have structure that supports exact calculations. There is no representation that is best for every task.

Source. Thrun, Burgard, and Fox, Probabilistic Robotics, early draft, Chapter 2, pp. 9–32 (PDF pp. 21–44).

Chapter 3: Gaussian Filters

The general Bayes filter updates an entire distribution. A Gaussian filter stores only a mean and covariance, making continuous-state estimation much more manageable. This is exact for linear Gaussian systems. For nonlinear systems, retaining a Gaussian is generally an approximation.

3.1 What a Gaussian belief represents

For $x \in \mathbb{R}^d$, write the belief as $\mathcal{N}(x; \mu, \Sigma)$. The MOMENTS REPRESENTATION consists of the d-component mean and the symmetric $d \times d$ covariance. It needs $d + d(d+1)/2$ independent numbers, so storage grows quadratically with state dimension. These two moments determine a Gaussian's higher moments; the higher moments are not all zero.

The covariance determines both spread and orientation. For example,

$$\Sigma = \begin{pmatrix} 4 & 3 \\ 3 & 4 \end{pmatrix}$$

describes positively correlated coordinate errors: an overestimate in one coordinate tends to accompany an overestimate in the other. Its principal directions are $(1, 1)$ and $(1, -1)$, with variances 7 and 1 respectively. Replacing it by the diagonal matrix $\mathrm{diag}(4, 4)$ loses that directional information.

A single Gaussian has one mode. It is suitable for tracking a location that is already approximately known, but cannot preserve two separated location hypotheses. An average between two plausible rooms might put the robot inside a wall. A large covariance does not turn a single peak into two peaks.

3.2 The linear Gaussian model

The Kalman filter assumes

$$x_t = A_t x_{t-1} + B_t u_t + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, R_t), \tag{3.1}$$

$$z_t = C_t x_t + \delta_t, \qquad \delta_t \sim \mathcal{N}(0, Q_t). \tag{3.2}$$

Here R_t is *motion* noise covariance and Q_t is *measurement* noise covariance, following the book's notation. For a d-dimensional state, m-dimensional control, and k-dimensional measurement:

$$A_t : d \times d, \quad B_t : d \times m, \quad C_t : k \times d, \quad R_t : d \times d, \quad Q_t : k \times k.$$

The initial belief must be Gaussian. The noise terms are zero-mean Gaussian, independent of the preceding state and of each other, with the corresponding independence across time. The controls are treated as known inputs. Under these assumptions, every prediction and posterior remains Gaussian.

3.3 The Kalman filter update

Given $(\mu_{t-1}, \Sigma_{t-1})$, perform the following calculation.

1. **Predict through the motion model.**

$$\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t, \tag{3.3}$$
$$\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t. \tag{3.4}$$

   The mean follows the nominal motion. The covariance is transformed by A_t on both sides because it describes products of coordinate deviations, and then gains the independent motion-noise covariance.

2. **Compare the reading with its prediction.**

$$r_t = z_t - C_t \bar{\mu}_t, \qquad S_t = C_t \bar{\Sigma}_t C_t^T + Q_t, \tag{3.5}$$
$$K_t = \bar{\Sigma}_t C_t^T S_t^{-1}. \tag{3.6}$$

   The INNOVATION r_t is the discrepancy in measurement coordinates. S_t is its covariance, including both state uncertainty projected into the sensor and the sensor's own noise. The KALMAN GAIN K_t converts this discrepancy into a state correction.

3. **Correct the belief.**

$$\mu_t = \bar{\mu}_t + K_t r_t, \tag{3.7}$$
$$\Sigma_t = (I - K_t C_t) \bar{\Sigma}_t. \tag{3.8}$$

   The covariance update accounts for information supplied by the measurement. It must use the same predicted covariance that was used to compute the gain.

For a scalar direct measurement ($C_t = 1$), with predicted variance P and measurement variance Q,

$$K = \frac{P}{P+Q}, \qquad \mu = (1-K)\bar{\mu} + Kz, \qquad \sigma^2 = \frac{PQ}{P+Q}.$$

A precise sensor has small Q and gets more weight. A precise prediction has small P and gets more weight instead. With $P, Q > 0$, the fused variance is smaller than either input variance: two independent clues constrain the same unknown more than either clue alone. This weighted-average interpretation applies directly to this scalar case; matrix gains are not scalar percentages.

3.3.1 Why these equations implement Bayes filtering

Prediction. An affine transformation of a Gaussian, plus independent Gaussian noise, is Gaussian. Taking expectation in (3.1) gives (3.3). Subtracting that mean gives

$$x_t - \bar{\mu}_t = A_t(x_{t-1} - \mu_{t-1}) + \varepsilon_t.$$

Multiply this expression by its transpose and take expectation. The cross terms vanish by independence and zero noise mean, leaving $A_t\Sigma_{t-1}A_t^T + R_t$. This proves the prediction covariance as well.

Correction. Suppress time subscripts and let $P = \bar{\Sigma}$. For this derivation, take P and Q to be positive definite. Multiplying the predicted Gaussian by the likelihood gives

$$b(x) \propto \exp[-J(x)], \qquad J(x) = \tfrac{1}{2}(x-\bar{\mu})^T P^{-1}(x-\bar{\mu}) + \tfrac{1}{2}(z-Cx)^T Q^{-1}(z-Cx).$$

This is a quadratic in x, so the posterior is Gaussian. Its curvature and stationary point determine its covariance and mean:

$$\Lambda := P^{-1} + C^T Q^{-1} C = \Sigma^{-1}, \tag{3.9}$$

$$\Lambda\mu = P^{-1}\bar{\mu} + C^T Q^{-1} z. \tag{3.10}$$

Subtract $\Lambda\bar{\mu}$ in the second equation to obtain

$$\mu - \bar{\mu} = \Sigma C^T Q^{-1}(z - C\bar{\mu}).$$

This already has the desired innovation form. To recover the computationally useful gain and covariance, let $S = CPC^T + Q$. Then

$$\Lambda P C^T = C^T Q^{-1} S, \qquad \text{hence} \qquad \Sigma C^T Q^{-1} = P C^T S^{-1} = K.$$

The corresponding inversion identity is

$$\Sigma = P - PC^T(CPC^T + Q)^{-1}CP = (I - KC)P. \tag{3.11}$$

To verify it, multiply its right-hand side by Λ:

$$\Lambda(P - PC^T S^{-1} CP) = I + C^T Q^{-1} CP - C^T Q^{-1} S S^{-1} CP = I.$$

This proves correction without explicitly inverting the state covariance. Equation (3.11) also shows that the covariance reduction $PC^T S^{-1} CP$ is positive semidefinite. Prediction need not always increase every variance, however: a contracting A_t can reduce uncertainty even before measurement noise is considered.

3.4 Worked example: one-dimensional tracking

Suppose a robot's location has mean 10 meters and variance 4 square meters. It commands a 2-meter displacement, with motion-noise variance 1. A direct position sensor then reports 14 meters with noise variance 4. These numbers are illustrative.

1. **Predict.** With $A = B = C = 1$,

$$\bar{\mu} = 10 + 2 = 12, \qquad P = 4 + 1 = 5.$$

2. **Calculate the discrepancy and gain.**

$$r = 14 - 12 = 2, \qquad S = 5 + 4 = 9, \qquad K = 5/9.$$

3. **Correct.**

$$\mu = 12 + (5/9)2 = \frac{118}{9} \approx 13.111, \qquad \sigma^2 = (1 - 5/9)5 = \frac{20}{9} \approx 2.222.$$

The reading pulls the estimate rightward, while the predicted uncertainty prevents treating 14 as exact. At the next time step, these posterior parameters become the previous belief; they are not replaced by the raw sensor reading.

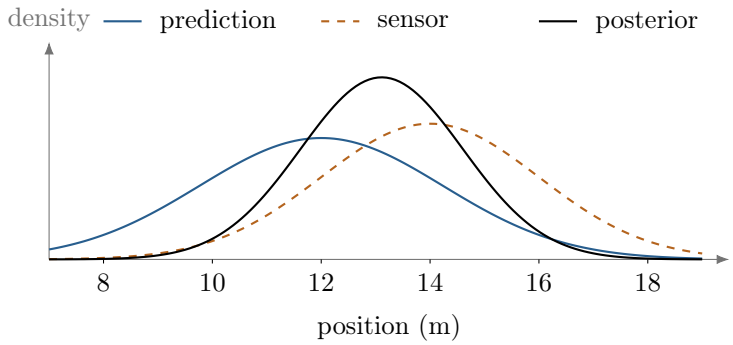


Figure 3: The worked example's Gaussian densities. The posterior is narrower and lies between the prediction and sensor reading. The sensor curve is the direct-measurement likelihood viewed as a function of position.

3.5 The extended Kalman filter

Real motion and sensing are often nonlinear:

$$x_t = g(u_t, x_{t-1}) + \varepsilon_t, \qquad z_t = h(x_t) + \delta_t.$$

For example, moving forward in the plane involves $\cos\theta$ and $\sin\theta$. Passing a Gaussian through such a function generally does not produce another Gaussian. The EXTENDED KALMAN FILTER (EKF) uses the function's tangent near the current estimate.

3.5.1 Linearize about the correct means

First-order Taylor expansion gives

$$g(u_t, x) \approx g(u_t, \mu_{t-1}) + G_t(x - \mu_{t-1}),$$
$$h(x) \approx h(\bar{\mu}_t) + H_t(x - \bar{\mu}_t),$$

where the JACOBIANS contain partial derivatives:

$$G_t = \left.\frac{\partial g(u_t, x)}{\partial x}\right|_{\mu_{t-1}}, \qquad H_t = \left.\frac{\partial h(x)}{\partial x}\right|_{\bar{\mu}_t}.$$

The motion Jacobian is evaluated at the previous mean, while the measurement Jacobian is evaluated at the *predicted* mean. Controls are fixed inputs when differentiating with respect to state.

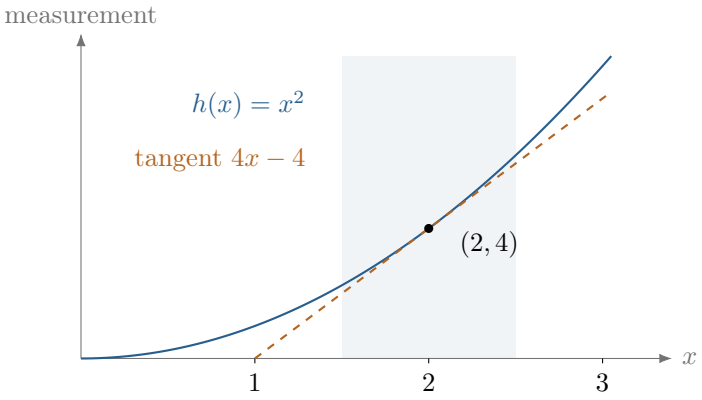


Figure 4: The tangent matches the function's value and slope at the mean. It is a local approximation: the discrepancy grows away from $x = 2$. The shaded band is an illustrative neighborhood, not a confidence region.

Substitute the affine approximations into the linear Gaussian calculation:

$$\bar{\mu}_t = g(u_t, \mu_{t-1}), \qquad \bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t, \tag{3.12}$$
$$S_t = H_t \bar{\Sigma}_t H_t^T + Q_t, \qquad K_t = \bar{\Sigma}_t H_t^T S_t^{-1}, \tag{3.13}$$
$$\mu_t = \bar{\mu}_t + K_t[z_t - h(\bar{\mu}_t)], \qquad \Sigma_t = (I - K_t H_t)\bar{\Sigma}_t. \tag{3.14}$$

Use the nonlinear functions to predict the mean and reading; use the Jacobians to propagate their local uncertainty. Replacing the predicted reading by $H_t\bar{\mu}_t$ would lose the affine offset and generally

be wrong. The derivation is the Kalman derivation applied to these local affine models, so the result is exact for the approximations, not for the original nonlinear system.

3.5.2 A nonlinear measurement and a failure case

Suppose the predicted scalar state has mean 2 and variance 1/4. The sensor measures $h(x) = x^2$ with noise variance 1, and reports $z = 5$. Then

$$H = 2\bar{\mu} = 4, \quad h(\bar{\mu}) = 4, \quad S = 4^2(1/4) + 1 = 5, \quad K = \frac{(1/4)4}{5} = \frac{1}{5}.$$

The EKF gives

$$\mu = 2 + \frac{1}{5}(5 - 4) = \frac{11}{5}, \qquad \sigma^2 = (1 - \tfrac{1}{5} \cdot 4)\frac{1}{4} = \frac{1}{20}.$$

This is a locally computed Gaussian estimate, not the exact posterior.

If instead $\bar{\mu} = 0$, the same sensor has $H = 0$, giving zero gain. The EKF then extracts no information even though a reading near 5 favors states near $+\sqrt{5}$ and $-\sqrt{5}$. A tangent at zero misses this two-sided constraint. Larger uncertainty exposes more curvature, so linearization quality depends on both the function and the belief's width. A small reported covariance does not prove that the approximation is accurate.

A Gaussian mixture, $\sum_j a_j \mathcal{N}(x; \mu_j, \Sigma_j)$ with $a_j \geq 0$ and $\sum_j a_j = 1$, can retain several hypotheses, at additional cost. Other approximations mentioned in the book include unscented methods and moment matching, which use more than the single tangent's local slope.

3.6 The information filter

The same nonsingular Gaussian can be stored in CANONICAL FORM:

$$\Omega = \Sigma^{-1}, \qquad \xi = \Omega\mu, \qquad \Sigma = \Omega^{-1}, \qquad \mu = \Omega^{-1}\xi.$$

Ω is the INFORMATION MATRIX, also called precision; ξ is the information vector. Expanding the Gaussian exponent gives

$$p(x) \propto \exp\left[-\tfrac{1}{2}x^T\Omega x + x^T\xi\right].$$

Terms independent of x are absorbed into the normalizer. Thus multiplying Gaussian factors adds their quadratic and linear coefficients. This is the reason measurements become especially simple in information form.

The centered quadratic $(x - \mu)^T\Omega(x - \mu)$ is the squared MAHALANOBIS DISTANCE: it measures deviation relative to the uncertainty, including correlations. A displacement along a high-variance direction is less surprising than the same displacement along a tightly constrained direction.

3.6.1 Prediction and correction

Substituting $\Sigma = \Omega^{-1}$ and $\mu = \Omega^{-1}\xi$ into the Kalman prediction gives

$$\bar{\Omega}_t = (A_t \Omega_{t-1}^{-1} A_t^T + R_t)^{-1}, \tag{3.15}$$
$$\bar{\xi}_t = \bar{\Omega}_t (A_t \Omega_{t-1}^{-1} \xi_{t-1} + B_t u_t). \tag{3.16}$$

For measurement $z_t = C_t x_t + \delta_t$, correction is additive:

$$\Omega_t = \bar{\Omega}_t + C_t^T Q_t^{-1} C_t, \tag{3.17}$$
$$\xi_t = \bar{\xi}_t + C_t^T Q_t^{-1} z_t. \tag{3.18}$$

To derive these, expand the log likelihood:

$$-\tfrac{1}{2}(z - Cx)^T Q^{-1}(z - Cx) = -\tfrac{1}{2} x^T C^T Q^{-1} C x + x^T C^T Q^{-1} z + \text{constant}.$$

Adding it to the predicted Gaussian's exponent gives exactly (3.17) and (3.18). The information filter and Kalman filter therefore encode the same posterior under the same linear Gaussian assumptions; they perform the bookkeeping differently.

For the tracking example, the predicted mean 12 and variance 5 give

$$\bar{\Omega} = \frac{1}{5}, \qquad \bar{\xi} = \frac{12}{5}.$$

The reading 14 with variance 4 contributes precision $1/4$ and information $14/4$. Hence

$$\Omega = \frac{1}{5} + \frac{1}{4} = \frac{9}{20}, \qquad \xi = \frac{12}{5} + \frac{14}{4} = \frac{59}{10}.$$

Recovering $\Sigma = 1/\Omega = 20/9$ and $\mu = \xi/\Omega = 118/9$ reproduces the Kalman result. Adding precision is the information-form version of combining two noisy estimates.

3.7 The extended information filter

The EXTENDED INFORMATION FILTER (EIF) applies the same Taylor approximations as the EKF, while storing canonical parameters. First recover μ_{t-1} by solving $\Omega_{t-1}\mu_{t-1} = \xi_{t-1}$. Prediction is

$$\bar{\mu}_t = g(u_t, \mu_{t-1}),$$
$$\bar{\Omega}_t = (G_t \Omega_{t-1}^{-1} G_t^T + R_t)^{-1},$$
$$\bar{\xi}_t = \bar{\Omega}_t \bar{\mu}_t.$$

The important detail is the measurement's affine offset. Linearization gives

$$z_t \approx h(\bar{\mu}_t) + H_t(x_t - \bar{\mu}_t) + \delta_t.$$

Move the known offset to the measurement side:

$$\widetilde{z}_t := z_t - h(\bar{\mu}_t) + H_t\bar{\mu}_t \quad \Longrightarrow \quad \widetilde{z}_t \approx H_t x_t + \delta_t.$$

Now apply the linear information update:

$$\Omega_t = \bar{\Omega}_t + H_t^T Q_t^{-1} H_t, \tag{3.19}$$

$$\xi_t = \bar{\xi}_t + H_t^T Q_t^{-1}[z_t - h(\bar{\mu}_t) + H_t\bar{\mu}_t]. \tag{3.20}$$

The sign before $H_t\bar{\mu}_t$ is positive. A useful check is the linear case $h(x) = Cx$: then $\widetilde{z} = z$, so the ordinary information update is recovered.

For the squared-measurement example, $\bar{\Omega} = 4$ and $\bar{\xi} = 8$. Since $H = 4$ and $\widetilde{z} = 5 - 4 + 4(2) = 9$,

$$\Omega = 4 + 4^2 = 20, \qquad \xi = 8 + 4(9) = 44.$$

Thus $\mu = 44/20 = 11/5$ and $\sigma^2 = 1/20$, agreeing with the EKF. Both filters share the approximation error; changing representation does not make the nonlinear posterior exactly Gaussian.

3.8 Computational and modeling tradeoffs

Moment and information forms have complementary strengths:

Operation	Moment form	Information form
Motion prediction	Direct mean/covariance propagation	Generally needs state-sized solves or inversions
Measurement update	Gain uses a measurement-sized solve	Add likelihood information to Ω and ξ
Retrieve mean	Already stored	Solve $\Omega\mu = \xi$

With dense matrices, general covariance prediction can take $O(d^3)$ time. A measurement update takes $O(d^2k + dk^2 + k^3)$ with standard dense algebra; small measurement dimension k or structured models can reduce the cost. Both representations use $O(d^2)$ storage when dense. These costs are polynomial in dimension, but can still be substantial for a large map.

In implementation, solve linear systems using suitable matrix factorizations rather than literally forming every displayed inverse. Covariances should remain symmetric and positive semidefinite; a negative variance is a numerical or modeling problem. Linearization and incorrectly independent noise models can make a filter overconfident even when the matrix arithmetic is sound.

Zero information, $\Omega = 0$ and $\xi = 0$, formally expresses a flat belief on an unbounded space, but it is an *improper* prior, not a normalized Gaussian with a recoverable mean. The inversion formulas above require enough information to make the relevant matrices nonsingular.

Independent likelihood contributions for the same state can be added in any order. In distributed sensing, however, shared prior information or reused measurements must not be counted twice, and delayed data must be associated with the correct state time. Addition alone does not remove these conditions.

A sparse information matrix can encode local conditional dependencies even when the covariance is dense. Exploiting that structure can change the cost substantially. The appropriate choice therefore depends on which operations are frequent, which variables each measurement touches, and the structure of the model, rather than on one representation being universally faster.

Source. Thrun, Burgard, and Fox, Probabilistic Robotics, early draft, Chapter 3, pp. 33–66 (PDF pp. 45–78).

Chapter 4: Nonparametric Filters

A Gaussian commits to one particular shape. Nonparametric filters instead describe a belief using cells or samples, and can represent several separated hypotheses. "Nonparametric" does not mean that nothing is stored: the resolution or number of samples is adjustable, rather than fixing the belief to a small family of shapes.

4.1 Discrete Bayes filters

For states $s_1, \dots, s_K$, store the probability vector $p_t = (p_{1,t}, \dots, p_{K,t})^T$. Define

$$T_{ki}(u_t) = P(X_t = s_k \mid X_{t-1} = s_i, u_t), \qquad L_k = p(z_t \mid X_t = s_k).$$

Columns of T sum to one: each column describes all destinations from one previous state. The Bayes recursion is

$$\bar{p}_{k,t} = \sum_{i=1}^{K} T_{ki}(u_t) p_{i,t-1}, \tag{4.1}$$

$$p_{k,t} = \frac{L_k \bar{p}_{k,t}}{\sum_{j=1}^{K} L_j \bar{p}_{j,t}}. \tag{4.2}$$

This is exact for the specified finite-state model. It is also the forward filtering recursion of a hidden Markov model. Compute the entire prediction from the *old* probability vector; overwriting old entries during the sum mixes two time steps.

4.1.1 A three-location example

Take an illustrative prior and transition matrix

$$p_{t-1} = \begin{pmatrix} 1/2 \\ 1/3 \\ 1/6 \end{pmatrix}, \qquad T = \begin{pmatrix} 0.8 & 0.1 & 0 \\ 0.2 & 0.8 & 0.2 \\ 0 & 0.1 & 0.8 \end{pmatrix}.$$

The transition allows mostly staying in place and some movement to a neighbor. For example, the first predicted probability is $0.8(1/2) + 0.1(1/3) + 0(1/6) = 13/30$. Suppose the observed landmark

has likelihoods $(1/10, 4/5, 1/5)$:

	s_1	s_2	s_3
prediction	13/30	2/5	1/6
likelihood	1/10	4/5	1/5
product	13/300	96/300	10/300
posterior	13/119	96/119	10/119

The products sum to 119/300. The reading favors the middle location, but does not erase the other possibilities.

Storing the belief takes $O(K)$ space. A dense transition needs $O(K^2)$ work per prediction. If each state has at most r possible destinations, sparse prediction needs $O(Kr)$ transition contributions instead.

4.2 Histograms for continuous states

A HISTOGRAM FILTER partitions a continuous state space into nonoverlapping cells $C_1, \dots, C_K$. Store cell *mass* $p_k = P(X \in C_k)$, not a density value. With cell volume $v_k = |C_k|$, the piecewise-uniform approximation is

$$b(x) = \frac{p_k}{v_k} \qquad \text{for } x \in C_k.$$

Integrating over the cell recovers p_k. This distinction matters for unequal cells: on the interval $[0, 3]$, uniform density $1/3$ gives mass $1/3$ to $[0, 1]$ and $2/3$ to $[1, 3]$, even though both cells have the same density.

4.2.1 Convert continuous models to cell models

Under the piecewise-uniform approximation, average the likelihood within a cell, and integrate motion over source and destination cells:

$$L_k = \frac{1}{v_k} \int_{C_k} p(z_t \mid x)\, dx,$$
$$T_{ki} = \frac{1}{v_i} \int_{C_i} \int_{C_k} p(x' \mid u_t, x)\, dx'\, dx.$$

Then use the discrete recursion (4.2). These integrals describe the cell model exactly under the uniform-within-cell assumption; they are not generally the exact continuous posterior after an update.

If the cells are small and the models vary little within them, representative centers c_i, c_k give the midpoint approximations

$$L_k \approx p(z_t \mid c_k), \qquad T_{ki} \approx v_k\, p(c_k \mid u_t, c_i).$$

The destination volume *multiplies* the transition density to produce probability mass. Equal volumes can cancel in a common normalization, but unequal volumes cannot be discarded. Boundary handling and numerical approximation must preserve valid transition probabilities; silently losing mass outside a modeled region changes the model.

4.2.2 Where to spend resolution

A fixed grid is straightforward, but resolution is expensive. Ten cells per coordinate require 10^d cells in d dimensions. Refining every region equally spends work on locations that may have almost no probability.

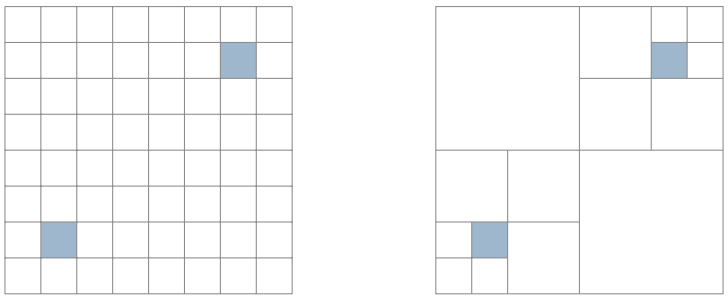


Figure 5: Schematic partitions for two regions of high belief. An adaptive tree retains fine cells near those regions and uses larger cells elsewhere. Shading identifies the regions, rather than specifying numerical masses.

DENSITY TREES recursively refine a partition where the current belief needs detail. SELECTIVE UPDATING keeps a fine grid but updates only a selected subset, while representing the remaining probability more coarsely. Both can save work, but dropping low-probability regions entirely risks losing hypotheses needed for recovery. Adaptive bookkeeping is also harder to implement than a fixed grid.

Topological representations use significant places, such as junctions, as states; metric representations retain numerical spatial relationships. This distinction concerns how space is modeled. Static versus adaptive partitioning concerns how a belief is discretized, so the two distinctions are not interchangeable.

4.3 Static binary state and log odds

Suppose an unknown binary fact x does not change, and successive readings are conditionally independent given that fact. An example is whether a fixed map cell is occupied, under the chosen static-world model. Let $p_t = P(x \mid z_{1:t})$ and define the LOG ODDS

$$\ell_t = \log \frac{p_t}{1 - p_t}, \qquad p_t = \frac{1}{1 + \exp(-\ell_t)}.$$

Use natural logarithms consistently here. Probability 1/2 corresponds to zero log odds; positive values favor x and negative values favor $\neg x$. The initial value is

$$\ell_0 = \log \frac{p_0}{1 - p_0}.$$

Dividing the Bayes update for x by the update for $\neg x$ cancels the normalizer:

$$\frac{p_t}{1 - p_t} = \frac{p(z_t \mid x)}{p(z_t \mid \neg x)} \frac{p_{t-1}}{1 - p_{t-1}}.$$

Taking logarithms turns repeated likelihood multiplication into addition.

Often an INVERSE MEASUREMENT MODEL directly supplies $q_t = P(x \mid z_t)$, computed using prior p_0, instead of describing all possible measurements through $p(z_t \mid x)$. Applying Bayes' rule once to this single-reading posterior gives

$$\frac{p(z_t \mid x)}{p(z_t \mid \neg x)} = \frac{q_t}{1 - q_t} \frac{1 - p_0}{p_0}.$$

Consequently,

$$\ell_t = \ell_{t-1} + \log \frac{q_t}{1 - q_t} - \log \frac{p_0}{1 - p_0}. \tag{4.3}$$

Subtracting the prior's log odds prevents counting that prior again with every inverse-model output. The inverse model must use the same reference prior assumed in this conversion.

For example, let $p_0 = 1/5$ and let each of two independent readings yield $q_t = 3/5$ under that prior. Initial odds are 1/4; each reading multiplies them by $(3/2)/(1/4) = 6$. The successive odds and probabilities are

	initial	one reading	two readings
odds	1/4	3/2	9
probability	1/5	3/5	9/10

This update is for a *static* state. A door that may open or close needs a transition prediction, as in Chapter 2. Likewise, repeatedly processing the same image is not repeated independent evidence. Exact probabilities zero or one give infinite log odds, so rounding to certainty can prevent recovery.

4.4 Particle beliefs and the bootstrap filter

A PARTICLE is a complete state hypothesis. A weighted set $\{(x^{[m]}, w^{[m]})\}_{m=1}^{M}$, with nonnegative weights summing to one, approximates expectations and region probabilities:

$$\mathbb{E}[\varphi(X)] \approx \sum_{m=1}^{M} w^{[m]} \varphi(x^{[m]}), \qquad P(X \in A) \approx \sum_{m=1}^{M} w^{[m]} \mathbf{1}\{x^{[m]} \in A\}.$$

The empirical distribution places mass at the particle locations; it is not an ordinary smooth density. Several clusters can preserve several location hypotheses. With equal weights, probability in a region is approximately the fraction of particles there.

The BOOTSTRAP PARTICLE FILTER starts with samples from $p(x_0)$. At each step it simulates motion, scores the resulting hypotheses against the measurement, then reallocates samples through RESAMPLING.

Algorithm 4.1: Bootstrap particle-filter update

Input: Equal-weight particles $\{x_{t-1}^{[m]}\}_{m=1}^{M}$, u_t, z_t
for $m = 1, \dots, M$ **do**
  $\widetilde{x}_t^{[m]} \sim p(x_t \mid u_t, x_{t-1}^{[m]})$
  $\widetilde{w}_t^{[m]} \leftarrow p(z_t \mid \widetilde{x}_t^{[m]})$
Normalize the weights: $w_t^{[m]} \leftarrow \widetilde{w}_t^{[m]} / \sum_j \widetilde{w}_t^{[j]}$
for $m = 1, \dots, M$ **do**
  Draw an index i_m with probability $w_t^{[i_m]}$
  $x_t^{[m]} \leftarrow \widetilde{x}_t^{[i_m]}$
return $\{x_t^{[m]}\}_{m=1}^{M}$, each with weight $1/M$

The draws in the last loop are with replacement, so a strong hypothesis can produce several copies. Resampling relocates computational effort toward likely states; it does not create information or make the approximation exact. The weighted set *before* resampling already represents the measurement update and can be used to estimate expectations directly.

Implementation needs a transition simulator and an evaluable measurement likelihood. It need not integrate the transition density over state space. If all weights are zero, resampling is undefined: check model support, numerical underflow, and whether the proposal has reached plausible states.

4.5 Why importance weights work

Suppose the desired TARGET DENSITY is f, but samples can only be drawn from a PROPOSAL DENSITY g. Assign each sample weight

$$\widetilde{w}^{[m]} = \frac{f(x^{[m]})}{g(x^{[m]})}, \qquad x^{[m]} \sim g.$$

The proposal must cover the target: $f(x) > 0$ requires $g(x) > 0$. Otherwise some target regions can never be represented, regardless of how weights are adjusted. The reason for the ratio is cancellation inside an expectation:

$$\mathbb{E}_g\left[\frac{f(X)}{g(X)}\varphi(X)\right] = \int g(x)\frac{f(x)}{g(x)}\varphi(x)\,dx = \mathbb{E}_f[\varphi(X)].$$

If f is known only up to a constant, normalize the sampled weights:

$$\mathbb{E}_f[\varphi(X)] \approx \frac{\sum_m \widetilde{w}^{[m]}\varphi(x^{[m]})}{\sum_m \widetilde{w}^{[m]}}.$$

This finite-sample ratio is generally biased. Under suitable integrability and finite-variance conditions, Monte Carlo errors have the familiar $M^{-1/2}$ scale, but the constant can be enormous when proposal and target barely overlap. This is convergence of probability or expectation estimates, not pointwise convergence of a smooth density made from isolated particles.

4.5.1 Why bootstrap weights are just likelihoods

For the derivation, temporarily consider a whole state path $x_{0:t}$. The Markov model gives the target factorization

$$p(x_{0:t} \mid z_{1:t}, u_{1:t}) \propto p(z_t \mid x_t)\, p(x_t \mid x_{t-1}, u_t) \cdot p(x_{0:t-1} \mid z_{1:t-1}, u_{1:t-1}).$$

If the previous paths already represent the preceding posterior and we extend them by sampling the motion model, the proposal contains the last two factors. Target divided by proposal therefore leaves only

$$\widetilde{w}_t \propto p(z_t \mid x_t).$$

Keeping the current endpoints of the weighted paths gives a representation of the current-state posterior. This explains the bootstrap algorithm's weight, while its finite particle set still introduces approximation error.

If resampling is skipped, previous particles are not reset to equal weights. Their weights must instead be propagated multiplicatively:

$$\widetilde{w}_t^{[m]} = w_{t-1}^{[m]} p(z_t \mid \widetilde{x}_t^{[m]}).$$

Reset to $1/M$ only *after* resampling. Resampling every step is a simple baseline, rather than a requirement of Bayes filtering.

4.6 Systematic resampling

Independent resampling can waste samples by chance. SYSTEMATIC RESAMPLING, called low-variance resampling in the book, uses one random offset and evenly spaced selection points. It operates on normalized weights.

Algorithm 4.2: Systematic resampling

Input: Particles $\{x^{[i]}\}_{i=1}^{M}$ and weights $w^{[i]}$ summing to one

Draw $r \sim \text{Uniform}[0, 1/M)$
$i \leftarrow 1$; $c \leftarrow w^{[1]}$
for $m = 1, \ldots, M$ **do**
    $u \leftarrow r + (m-1)/M$
    while $u \geq c$ **do**
        $i \leftarrow i + 1$
        $c \leftarrow c + w^{[i]}$
    Output a copy of $x^{[i]}$ with weight $1/M$

The cumulative-weight intervals have lengths $w^{[i]}$. The points select the interval containing them. Using the first cumulative value strictly greater than u handles zero weights and exact boundaries consistently; either boundary convention has the same law for a continuous random offset.

Although there is a nested loop, the pointer i only moves forward, and crosses the particle list at most once. The whole operation takes $O(M)$ time. The offspring counts are dependent, but each particle's expected number of copies is $Mw^{[i]}$. With equal weights, every particle is copied once, so resampling does not randomly destroy diversity in that case.

4.6.1 A complete four-particle execution

Suppose motion simulation produces locations $(1, 2, 5, 6)$, and measurement likelihoods proportional to $(0.1, 0.2, 0.6, 0.1)$. These already sum to one:

particle index	1	2	3	4
location	1	2	5	6
weight	0.1	0.2	0.6	0.1
cumulative weight	0.1	0.3	0.9	1

Choose illustrative offset $r = 0.08$, which lies in $[0, 1/4)$. The selection points are $0.08, 0.33, 0.58, 0.83$. The first lies in interval 1; the remaining three lie in interval 3. Thus the new equal-weight set is $(1, 5, 5, 5)$.

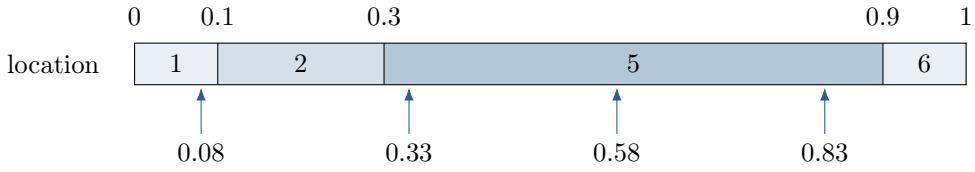


Figure 6: Systematic resampling for the worked example. Interval length is weight; equally spaced arrows select one copy of location 1 and three of location 5. The single random offset shifts all arrows together.

Before resampling, the weighted mean is $0.1(1) + 0.2(2) + 0.6(5) + 0.1(6) = 4.1$. After this particular draw, the empirical mean is $(1+5+5+5)/4 = 4$. The difference is sampling error, not another sensor update. More particles reduce typical error, but a resampled set is never guaranteed to preserve every statistic of the weighted set exactly.

4.7 Failure modes and resource choices

Four limitations explain why a particle filter can fail despite a simple implementation.

1. **Too few particles.** With $M = 1$ and a positive likelihood, the only normalized weight is one. Resampling must retain that particle, so the current reading has no effect on the selected state. A finite cloud is not a collection of exact independent posterior samples.

2. **Resampling noise and lost diversity.** Consider a static state with no informative readings. Repeated independent resampling can eventually leave only copies of one initial hypothesis, despite no information having arrived. This is sampling-induced confidence. Resample less often when weights remain balanced, retain weights between such steps, or use a more systematic scheme. A stationary robot can still learn from genuinely new, conditionally independent observations; repeated correlated observations need a model that accounts for their dependence.

3. **Proposal–target mismatch.** Motion samples may be spread over a large region while a precise sensor accepts only a tiny part of it. Most weights then become negligible. If the observation imposes an exact lower-dimensional constraint, a continuous motion proposal may miss it entirely. A measurement-informed proposal can help; merely increasing sensor noise changes the inference model and is an approximation, not an exact fix.

4. **Loss of a relevant mode.** If no particle remains near the true state, weighting alone cannot restore that hypothesis. For 100 independent draws from a distribution, a region with probability 0.01 is missed with probability $(0.99)^{100} \approx 0.366$. There is no universally safe particle count. High dimension and broad uncertainty make coverage especially hard. Unweighted injection of arbitrary new particles changes the represented posterior; recovery mechanisms should be modeled or weighted accordingly.

Adaptive methods spend more cells or particles when the belief is broad or complex, and fewer when it is concentrated. Resource-adaptive methods also respect the available computation time. The appropriate budget depends on state dimension, the number of plausible modes, the motion model, and sensor precision. More samples cannot compensate for incorrect independence assumptions or a proposal with missing support.

Source. Thrun, Burgard, and Fox, Probabilistic Robotics, early draft, Chapter 4, pp. 67–90 (PDF pp. 79–102).